

Comprehensive Guide: Core Placement Optimization for Multi-Chip Neural Network Systems

Executive Summary

This document provides an exhaustive architectural, mathematical, and programmatic breakdown of the Core Placement Optimization system using Reinforcement Learning (RL). The system is designed to map Neural Network workloads onto Multi-Chip Many-Core hardware architectures. Built upon the principles outlined in the 2020 ACM paper "Core Placement Optimization for Multi-chip Many-core Neural Network Systems with Reinforcement Learning," this guide details the transition from discrete Proximal Policy Optimization (PPO) algorithms to continuous Deep Deterministic Policy Gradient (DDPG) agents, the mathematical modeling of non-uniform network-on-chip (NoC) topologies, and the integration of PyTorch forward hooks to extract dynamic communication volumes from real-world Convolutional Neural Networks (CNNs).

1. The Core Placement Problem

1.1 The Hardware: Multi-Chip Modules (MCMs)

As deep neural networks scale, single-chip processors face fundamental silicon yield limits. The industry has shifted towards Multi-Chip Modules (MCMs) or "chiplets," where multiple smaller silicon dies are connected via an interposer or substrate to act as a single logical processor. While this improves manufacturing yields, it introduces a severe communication bottleneck:

Non-Uniform Memory Access (NUMA).

- **On-Chip Links:** Wires connecting cores inside the exact same silicon die. These links provide massive bandwidth and ultra-low latency.
- **Off-Chip Links:** Wires that cross the physical gap between two separate chips. These links have significantly lower bandwidth and much higher latency (often 5x to 10x slower).

1.2 The Software: Task Graphs

A neural network is essentially a massive Directed Acyclic Graph (DAG) of computations. During inference or training, tensor data (activations) flows sequentially from one layer to the next. In our system, each layer (or sub-layer operation) is defined as a "Task." The size of the tensor passed between Task A and Task B is the "Communication Volume."

1.3 The Goal

The core placement problem asks: **Which physical hardware core should execute which software task?** If two tasks share a massive communication volume, placing them on separate chips will force gigabytes of data to cross the slow off-chip links, stalling the entire neural

network. The combinatorial space for mapping N tasks to N cores is $N!$ (factorial). For a 64-core system, there are roughly 1.26×10^{89} possible configurations, making exhaustive search mathematically impossible and traditional heuristic methods (like Simulated Annealing) extremely slow for dynamic workloads.

2. Hardware Topology & Mathematical Modeling

The physics of the multi-chip system are modeled in the `multi_chip_topology.py` file. The code maps 1D task IDs to 2D physical coordinates to compute spatial distances.

2.1 The Coordinate System

A multi-chip grid consists of X chips horizontally and Y chips vertically. Each chip contains a grid of cores (R rows by C columns). To find the distance between Core A and Core B, the system resolves their global grid coordinates:

```
def core_xy_global(self, core_id: int):
    chip_id = core_id // self.cores_per_chip
    local_core = core_id % self.cores_per_chip
    chip_x = chip_id % self.num_chips_x
    chip_y = chip_id // self.num_chips_x
    local_c = local_core % self.cols_per_chip
    local_r = local_core // self.cols_per_chip
    return chip_x, chip_y, local_r, local_c
```

2.2 The Hierarchical Latency Function

Data travels in a 2D mesh network using Manhattan Routing (moving exclusively along the X and Y axes). The latency incorporates the hierarchical penalties proposed in the 2020 ACM paper:

- **α (alpha):** The baseline latency cost to hop one core over on an on-chip link (Default: 1.0).
- **β (beta):** The massive penalty latency cost to cross a boundary to an adjacent chip (Default: 5.0).

The mathematical distance $L(A, B)$ is calculated as:

```
On-Chip Hops = |Row_A - Row_B| + |Col_A - Col_B|
Off-Chip Hops = |ChipX_A - ChipX_B| + |ChipY_A - ChipY_B|
Latency(A, B) = (On-Chip Hops *  $\alpha$ ) + (Off-Chip Hops *  $\beta$ )
```

3. The Optimization Objective (Environment)

The Reinforcement Learning agent acts inside the `multi_chip_environment.py`. The environment calculates the total energy and latency cost of a placement to generate the reward signal.

3.1 Total Communication Cost

The absolute cost of a placement configuration is the sum of the data volume traveling over the wires multiplied by the physical latency of those wires. For N tasks, the objective function is:
$$\text{Total Cost} = \sum (\text{Volume}[i, j] * \text{Latency}[\text{Core}_i, \text{Core}_j]) \text{ for all task pairs } (i, j)$$

Because the Reinforcement Learning agent attempts to *maximize* its reward, the environment returns the negative of the cost as the reward: $\text{Reward} = -\text{Total Cost}$.

4. The Reinforcement Learning Engine: DDPG

The transition from Proximal Policy Optimization (PPO) to Deep Deterministic Policy Gradient (DDPG) in `run_multi_chip.py` is the system's most critical algorithmic upgrade.

4.1 Why DDPG? Continuous vs. Discrete Action Spaces

PPO operates in discrete action spaces. For a 64-core system, a discrete PPO agent outputs a probability distribution over 64 distinct classes. As the board fills up, predicting the exact remaining valid core IDs becomes sparse and numerically unstable. DDPG, however, operates in a **continuous action space**. The Actor network outputs a 2D physical coordinate (X, Y) normalized between $[-1, 1]$. This allows the agent to learn "spatial regions" (e.g., "I should place this task in the top-left corner") rather than memorizing discrete arbitrary IDs.

4.2 The Actor-Critic Architecture

The DDPG implementation utilizes two distinct Multi-Layer Perceptron (MLP) neural networks:

- **The Actor:** Observes the state of the board and outputs the deterministic continuous coordinates $[-1.0, 1.0]$ for the next placement.
- **The Critic:** Acts as a judge. It takes both the current state AND the Actor's proposed action as inputs, and outputs a single Q-Value estimating how good that placement will be in the long run.

4.3 State Representation

The agent must "see" the board. The state is a concatenated 1D array containing:

1. The physical occupancy map (1.0 if a core is taken, 0.0 if empty).
2. The communication volume array of the *current* task waiting to be placed. This allows the agent to know which previously placed tasks it needs to be near.

5. The Nearest-Neighbor Collision Resolution

In previous iterations, if the DDPG agent's continuous coordinate pointed to a core that already had a task on it, the system would pick a random empty core. This destroyed the agent's gradient signal. Following the 2020 paper's exact pseudocode, the system now implements

Nearest-Neighbor resolution.

When the Actor outputs (A_x, A_y) , it is scaled to the grid dimensions $(Target_X, Target_Y)$. The system calculates the Euclidean distance to all *unoccupied* cores on the board:

$$Distance = (Core_X - Target_X)^2 + (Core_Y - Target_Y)^2$$

The task is assigned to the available core with the absolute minimum distance. This guarantees that the agent's spatial intent is preserved perfectly, solving the DDPG convergence issue on crowded grid boards.

6. Disambiguating "CNN": The Workload Extractor

A major point of confusion in the codebase is the acronym "CNN". The 2020 ACM paper uses a CNN (Convolutional Neural Network) as the brain of the RL agent to process the 2D grid visually. Our implementation uses an MLP for the agent's brain to remain lightweight. However, we introduced a **CNN Workload Extractor** to generate realistic input data.

6.1 Real-World Data vs. Random Math

Originally, the task graph was populated with `np.random`. To test the RL agent properly, we integrated a real 15-layer PyTorch CNN (image classifier) to serve as the benchmark workload.

6.2 PyTorch Forward Hooks

To measure the exact communication volume traveling over the wires, the code uses PyTorch Forward Hooks. When a dummy 32×32 image is passed through the network, the hooks intercept the output tensors at every layer.

```
def hook_fn(module, input, output):  
    layer_output_sizes.append(output.numel())
```

If Layer 1 (Conv2D) outputs a tensor of shape [Batch=1, Channels=16, Height=32, Width=32], the hook calculates the total elements: $1 * 16 * 32 * 32 = 16,384$ parameters. The workload matrix is updated so that `Task_Graph[0, 1] = 16384`. This forces the RL agent to physically cluster these highly-communicating layers together to avoid massive off-chip penalties.

7. Comparison & Analysis

Feature	2020 ACM Paper Implementation	Our Final Implementation
Hardware Topology	Hierarchical Mesh (alpha/beta latencies)	Perfect Match (<code>multi_chip_topology.py</code>)
Optimization Objective	Minimize Vol * Dist	Perfect Match (<code>multi_chip_environment.py</code>)
RL Algorithm	Deep Deterministic Policy Gradient (DDPG)	Perfect Match (<code>run_multi_chip.py</code>)
Collision Resolution	Euclidean Nearest Unoccupied	Perfect Match

Feature	2020 ACM Paper Implementation	Our Final Implementation
	Neighbor	
Agent Architecture	Convolutional Neural Networks (CNN)	Multi-Layer Perceptrons (MLP) for CPU compatibility
Workload	Real DNN/CNN benchmark traces	Live dynamic extraction via PyTorch hooks

8. Future Work & Enhancements

While the current pipeline successfully demonstrates the superiority of continuous control (DDPG) over discrete RL (PPO) for placement tasks, the ultimate goal is to bridge the final architectural gap to match the state-of-the-art fully.

1. **Spatial Agents:** Upgrading the Actor and Critic networks from `nn.Linear` to `nn.Conv2d`. This will allow the agent to treat the placement grid as an image, naturally extracting spatial clusters and further accelerating training time.
2. **Graph Neural Network Embeddings:** Instead of feeding just the current task's communication volume into the state vector, embedding the entire DAG using a Graph Convolutional Network (GCN) would allow the agent to anticipate bottlenecks deeper in the pipeline before placing the first layers.